12
Image Generation Using GANs

In the previous chapter, we learned about manipulating an image using neural style transfer and super-imposed the expression in one image on another. However, what if we give the network a bunch of images and ask it to come up with an entirely new image, all on its own?

Generative adversarial networks (GANs) are a step toward achieving the feat of generating an image given a collection of images. In this chapter, we will start by learning about the idea behind what makes GANs work, before building one from scratch. This is a vast field that is expanding even as we write this book. This chapter will lay the foundation of GANs by covering three variants; we will learn about more advanced GANs and their applications in the next chapter.

In this chapter, we will explore the following topics:

- Introducing GANs
- Using GANs to generate handwritten digits
- Using DCGANs to generate face images
- Implementing conditional GANs

All code snippets within this chapter are available in the `Chapter12` folder of the Github repository at https://bit.ly/mcvp-2e.

Introducing GANs

To understand GANs, we need to understand two terms: generator and discriminator. First, we should have a reasonable sample of images (100-1000 images) of an object. A **generative network** (generator) learns rep-

resentation from a sample of images and then generates images similar to the sample of images. A **discriminator network** (discriminator) is one that looks at the image generated by the generator network and the original sample of images and classifies images as original or generated (fake) ones.

The generator network tries to generate images in such a way that the discriminator classifies the images as real. The discriminator network tried to classify the generated images as fake and the images in the original sample as real.

Essentially, the adversarial term in GAN represents the opposite nature of the two networks—*a generator network, which generates images to fool the discriminator network, and a discriminator network that classifies each image by saying whether the image is generated or is an original.*

Let's understand the process employed by GANs through the following diagram:

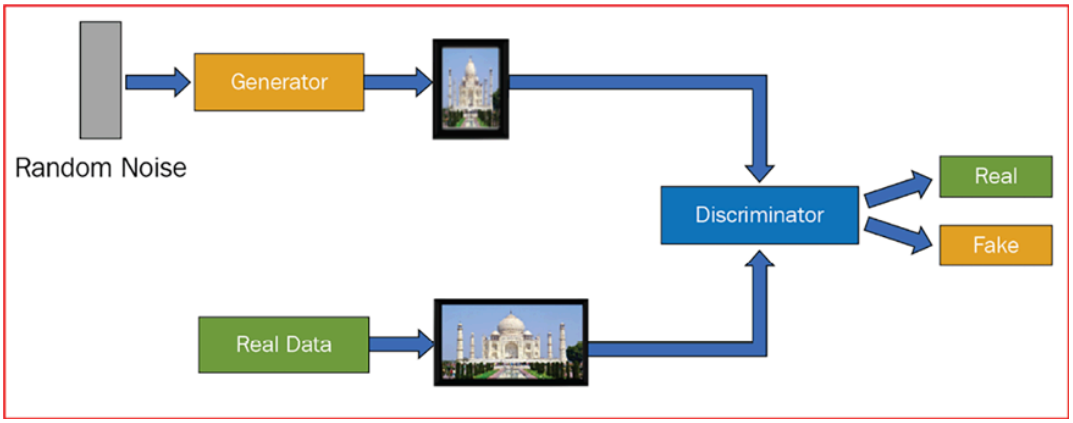


Figure 12.1: Typical GAN workflow

In the preceding diagram, the generator network is generating images from random noise as input. A discriminator network is looking at the images generated by the generator and comparing them with real data (a sample of images that are provided) to specify whether the generated image is real or fake. The generator tries to generate as many realistic images as possible, while the discriminator tries to detect which of the images that are generated by the generator are fake. This way, the generator

learns to generate as many realistic images as possible by learning from what the discriminator looks at to identify whether an image is fake.

Typically, the generator and discriminator are trained alternately within each step of training. This way, it becomes a cops-and-robbers game, where the generator is the robber trying to generate fake data, while the discriminator is the cop trying to identify the available data as real or fake. The steps involved in training GANs are as follows:

1. Train the generator (and not the discriminator) to generate images such that the discriminator classifies the images as real.
2. Train the discriminator (and not the generator) to classify the images that the generator generates as fake.
3. Repeat the process until an equilibrium is achieved.

Let's now understand how we compute the loss values for both the generator and discriminator to train both the networks together using the following diagram and steps:

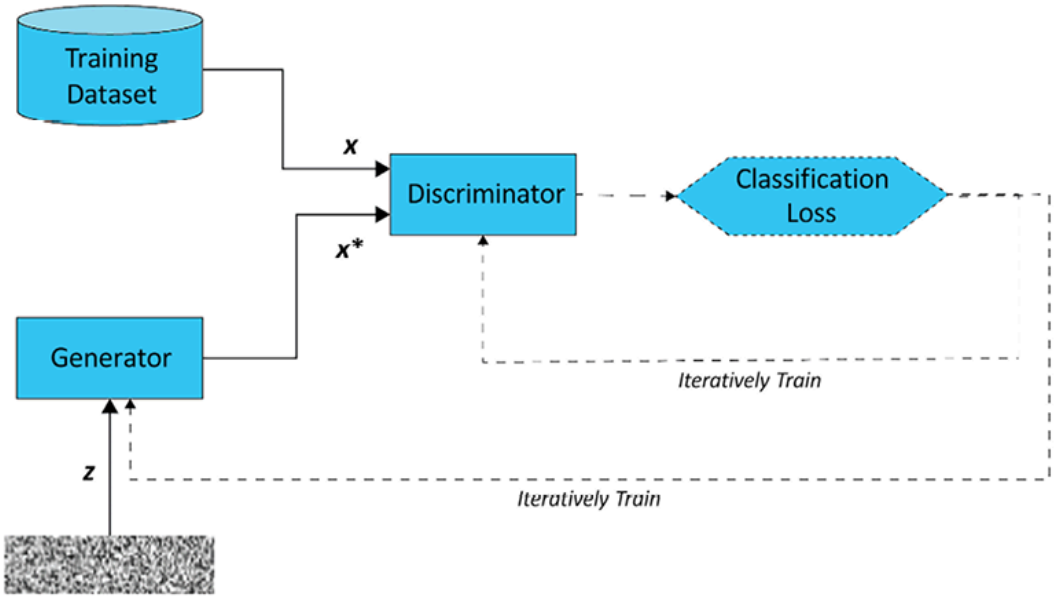


Figure 12.2: GAN training workflow (dotted lines represent training, solid lines process)

In the preceding scenario, when the discriminator can detect generated images really well, the loss corresponding to the generator is much

higher when compared to the loss corresponding to the discriminator. Thus, the gradients adjust in such a way that the generator would have a lower loss. However, it would tip the discriminator loss to a higher side. In the next iteration, the gradients adjust so that the discriminator loss is lower. This way, the generator and discriminator keep getting trained until a point where the generator generates realistic images and the discriminator cannot distinguish between a real or a generated image.

With this understanding, let's generate images relating to the MNIST dataset in the next section.

Using GANs to generate handwritten digits

To generate images of handwritten digits, we will leverage the same network as we learned about in the previous section. The strategy we will adopt is as follows:

1. Import MNIST data.
2. Initialize random noise.
3. Define the generator model.
4. Define the discriminator model.
5. Train the two models alternately.
6. Let the model train until the generator and discriminator losses are largely the same.

Let's execute each of the preceding steps in the following code:

The following code is available as `Handwritten_digit_generation_using_GAN.ipynb` in the `Chapter12` folder in this book's GitHub repository: **https://bit.ly/mcvp-2e**. The code is moderately lengthy. We strongly recommend you execute the notebook in GitHub to reproduce the results while you understand the steps to perform and the explanation of various code components from the text.

1. Import the relevant packages and define the device:

```
!pip install -q torch_snippets
from torch_snippets import *
device = "cuda" if torch.cuda.is_available() else "cpu"
from torchvision.utils import make_grid
```

2. Import the `MNIST` data and define the dataloader with built-in data transformation so that the input data is scaled to a mean of 0.5 and a standard deviation of 0.5:

```
from torchvision.datasets import MNIST
from torchvision import transforms
transform = transforms.Compose([
                transforms.ToTensor(),
                transforms.Normalize(mean=(0.5,), std=(0.5,))
        ])
data_loader = torch.utils.data.DataLoader(MNIST('~/data', \
                        train=True, download=True, transform=transform), \
                        batch_size=128, shuffle=True, drop_last=True)
```

3. Define the `Discriminator` model class:

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
                                nn.Linear(784, 1024),
                                nn.LeakyReLU(0.2),
                                nn.Dropout(0.3),
                                nn.Linear(1024, 512),
                                nn.LeakyReLU(0.2),
                                nn.Dropout(0.3),
                                nn.Linear(512, 256),
                                nn.LeakyReLU(0.2),
                                nn.Dropout(0.3),
                                nn.Linear(256, 1),
                                nn.Sigmoid()
```

```
                                    )
    def forward(self, x): return self.model(x)
```

A summary of the discriminator network is as follows:

Note that, in the preceding code, in place of `ReLU`, we have used `LeakyReLU` (an empirical evaluation of different types of ReLU activations is available here: https://arxiv.org/pdf/1505.00853) as the activation function.

```
!pip install torch_summary
from torchsummary import summary
discriminator = Discriminator().to(device)
summary(discriminator,torch.zeros(1,784))
```

The preceding code generates the following output:

```
==========================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================
├─Sequential: 1-1                        [-1, 1]                   --
|    └─Linear: 2-1                       [-1, 1024]                803,840
|    └─LeakyReLU: 2-2                    [-1, 1024]                --
|    └─Dropout: 2-3                      [-1, 1024]                --
|    └─Linear: 2-4                       [-1, 512]                 524,800
|    └─LeakyReLU: 2-5                    [-1, 512]                 --
|    └─Dropout: 2-6                      [-1, 512]                 --
|    └─Linear: 2-7                       [-1, 256]                 131,328
|    └─LeakyReLU: 2-8                    [-1, 256]                 --
|    └─Dropout: 2-9                      [-1, 256]                 --
|    └─Linear: 2-10                      [-1, 1]                   257
|    └─Sigmoid: 2-11                     [-1, 1]                   --
==========================================================================
Total params: 1,460,225
Trainable params: 1,460,225
Non-trainable params: 0
Total mult-adds (M): 2.92
```

Figure 12.3: Summary of the discriminator architecture

4. Define the `Generator` model class:

```
class Generator(nn.Module):
    def __init__(self):
```

```
        super().__init__()
        self.model = nn.Sequential(
                                nn.Linear(100, 256),
                                nn.LeakyReLU(0.2),
                                nn.Linear(256, 512),
                                nn.LeakyReLU(0.2),
                                nn.Linear(512, 1024),
                                nn.LeakyReLU(0.2),
                                nn.Linear(1024, 784),
                                nn.Tanh()
                            )
    def forward(self, x): return self.model(x)
```

Note that the generator takes a 100-dimensional input (which is of random noise) and generates an image from the input. A summary of the generator model is as follows:

```
generator = Generator().to(device)
summary(generator,torch.zeros(1,100))
```

The preceding code generates the following output:

```
==========================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================
├─Sequential: 1-1                        [-1, 784]                 --
|    └─Linear: 2-1                       [-1, 256]                 25,856
|    └─LeakyReLU: 2-2                    [-1, 256]                 --
|    └─Linear: 2-3                       [-1, 512]                 131,584
|    └─LeakyReLU: 2-4                    [-1, 512]                 --
|    └─Linear: 2-5                       [-1, 1024]                525,312
|    └─LeakyReLU: 2-6                    [-1, 1024]                --
|    └─Linear: 2-7                       [-1, 784]                 803,600
|    └─Tanh: 2-8                         [-1, 784]                 --
==========================================================================
Total params: 1,486,352
Trainable params: 1,486,352
Non-trainable params: 0
Total mult-adds (M): 2.97
```

Figure 12.4: Summary of the generator architecture

5. Define a function to generate random noise and register it to the device:

```
def noise(size):
    n = torch.randn(size, 100)
    return n.to(device)
```

6. Define a function to train the discriminator, as follows:
    1. The discriminator training function (`discriminator_train_step`) takes real data (`real_data`) and fake data (`fake_data`) as input:

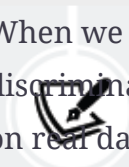
```
def discriminator_train_step(real_data, fake_data):
```

    2. Reset the gradients:

```
        d_optimizer.zero_grad()
```

    3. Predict on the real data (`real_data`) and calculate the loss (`error_real`) before performing backpropagation on the loss value:

```
        prediction_real = discriminator(real_data)
        error_real = loss(prediction_real, \
                          torch.ones(len(real_data),1).to(device))
        error_real.backward()
```



When we calculate the discriminator loss on real data, we expect the discriminator to predict an output of `1` . Hence, the discriminator loss on real data is calculated by expecting the discriminator to predict the output to be `1` using `torch.ones` during discriminator training.

    4. Predict on the fake data (`fake_data`) and calculate the loss (`error_fake`) before performing backpropagation on the loss value:

```
        prediction_fake = discriminator(fake_data)
        error_fake = loss(prediction_fake, \
                    torch.zeros(len(fake_data),1).to(device))
        error_fake.backward()
```

When we calculate the discriminator loss on fake data, we expect the discriminator to predict an output of `0`. Hence, the discriminator loss on fake data is calculated by expecting the discriminator to predict an output of `0` using `torch.zeros` during discriminator training.

5. Update the weights and return the overall loss (summing up the loss values of `error_real` on `real_data` and `error_fake` on `fake_data`):

```
        d_optimizer.step()
        return error_real + error_fake
```

7. Train the generator model in the following way:
    1. Define the generator training function (`generator_train_step`) that takes fake data (`fake_data`):

```
def generator_train_step(fake_data):
```

    2. Reset the gradients of the generator optimizer:

```
    g_optimizer.zero_grad()
```

    3. Predict the output of the discriminator on fake data (`fake_data`):

```
    prediction = discriminator(fake_data)
```

    4. Calculate the generator loss value by passing `prediction` and the expected value as `torch.ones` since we want to fool the discriminator to output a value of `1` when training the generator:

```
    error = loss(prediction, torch.ones(len(real_data),1).to(device))
```

    5. Perform backpropagation, update the weights, and return the error:

```
    error.backward()
    g_optimizer.step()
    return error
```

8. Define the model objects, the optimizer for each generator and discriminator, and the loss function to optimize:

```
discriminator = Discriminator().to(device)
generator = Generator().to(device)
d_optimizer= optim.Adam(discriminator.parameters(),lr=0.0002)
g_optimizer = optim.Adam(generator.parameters(), lr=0.0002)
loss = nn.BCELoss()
num_epochs = 200
log = Report(num_epochs)
```

9. Run the models over increasing epochs:
    1. Loop through 200 epochs (`num_epochs`) over the `data_loader` function obtained in *step 2*:

```
for epoch in range(num_epochs):
    N = len(data_loader)
    for i, (images, _) in enumerate(data_loader):
```

    2. Load real data (`real_data`) and fake data, where fake data (`fake_data`) is obtained by passing `noise` (with a batch size of the number of data points in `real_data`: `len(real_data)`) through the `generator` network. Note that it is important to run `fake_data.detach()`, or else training will not work. On detaching, we are creating a fresh copy of the tensor so that when `error.backward()` is called in `discriminator_train_step`, the tensors associated with the generator (which create `fake_data`) are not affected:

```
        real_data = images.view(len(images), -1).to(device)
        fake_data=generator(noise(len(real_data))).to(device)
        fake_data = fake_data.detach()
```

3. Train the discriminator using the `discriminator_train_step` function defined in *step 6*:

```
        d_loss=discriminator_train_step(real_data, fake_data)
```

4. Now that we have trained the discriminator, let's train the generator in this step. Generate a new set of fake images (`fake_data`) from noisy data and train the generator using `generator_train_step` defined in *step 6*:

```
        fake_data=generator(noise(len(real_data))).to(device)
        g_loss = generator_train_step(fake_data)
```

5. Record the losses:

```
        log.record(epoch+(1+i)/N, d_loss=d_loss.item(), \
                   g_loss=g_loss.item(), end='\r')
    log.report_avgs(epoch+1)
log.plot_epochs(['d_loss', 'g_loss'])
```

The discriminator and generator losses over increasing epochs are as follows (you can refer to the digital version of the book for the colored image):

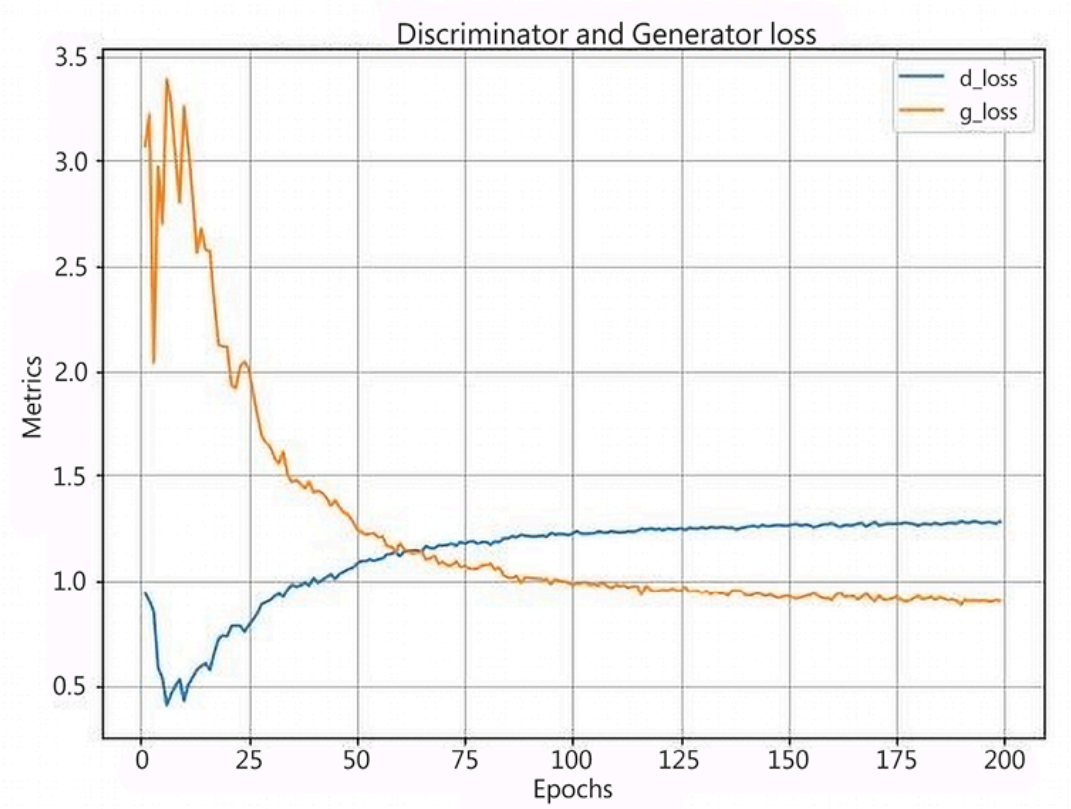


Figure 12.5: Discriminator and Generator loss over increasing epochs

10. Visualize the fake data post-training:

```
z = torch.randn(64, 100).to(device)
sample_images = generator(z).data.cpu().view(64, 1, 28, 28)
grid = make_grid(sample_images, nrow=8, normalize=True)
show(grid.cpu().detach().permute(1,2,0), sz=5)
```

The preceding code generates the following output:

Figure 12.6: Generated digits

From this, we can see that we can leverage GANs to generate images that are realistic, but still with some scope for improvement. In the next section, we will learn about using deep convolutional GANs to generate more realistic images.

Using DCGANs to generate face images

In the previous section, we learned about generating images using GANs. However, we have already seen in *Chapter 4*, *Introducing Convolutional Neural Networks*, that Convolutional Neural Networks (CNNs) perform better in the context of images when compared to vanilla neural networks. In this section, we will learn about generating images using Deep Convolutional Generative Adversarial Networks (DCGANs), which use convolution and pooling operations in the model.

First, let's understand the technique we will leverage to generate an image using a set of 100 random numbers (we chose 100 random numbers

so that the network has a reasonable number of values to generate images. We encourage readers to experiment with different amounts of random numbers and see the result). We will first convert noise into a shape of *batch size x 100 x 1 x 1*.

The reason for appending additional channel information in DCGANs and not doing it in the GAN section is that we will leverage CNNs in this section, which requires inputs in the form of batch size x channels x height x width.

Next, we convert the generated noise into an image by leveraging

`ConvTranspose2d`. As we learned in *Chapter 9*, *Image Segmentation*, `ConvTranspose2d` does the opposite of a convolution operation, which is to take input with a smaller feature map size (height x width) and upsample it to that of a larger size using a predefined kernel size, stride, and padding. This way, we would gradually convert a vector from a shape of batch size x 100 x 1 x 1 into a shape of batch size x 3 x 64 x 64. With this, we have taken a random noise vector of size 100 and converted it into an image of a face.

With this understanding, let's now build a model to generate images of faces:

The following code is available as `Face_generation_using_DCGAN.ipynb` in the `Chapter12` folder in this book's GitHub repository: **https://bit.ly/mcvp-2e**. The code contains URLs to download data from and is moderately lengthy. We strongly recommend you execute the notebook in GitHub to reproduce the results while you understand the steps to perform and the explanation of various code components from the text.

1. Download and extract the face images (a dataset that we have collated by generating faces of random people):

```
!wget https://www.dropbox.com/s/rbajpdlh7efkdo1/male_female_face_images.zi
!unzip male_female_face_images.zip
```

A sample of the images is shown here:

Figure 12.7: Sample male and female images

2. Import the relevant packages:

```
!pip install -q --upgrade torch_snippets
from torch_snippets import *
import torchvision
from torchvision import transforms
import torchvision.utils as vutils
import cv2, numpy as np, pandas as pd
device = "cuda" if torch.cuda.is_available() else "cpu"
```

3. Define the dataset and dataloader:
    1. Ensure that we crop the images so that we retain only the faces and discard additional details in the image. First, we will download the cascade filter (more on cascade filters can be found in the *Using OpenCV Utilities for Image Analysis* PDF on GitHub), which will help with identifying faces within an image:

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + \
                        'haarcascade_frontalface_default.xml')
```

2. Create a new folder and dump all the cropped face images into the new folder:

```
!mkdir cropped_faces
images = Glob('/content/females/*.jpg') + \
            Glob('/content/males/*.jpg')
for i in range(len(images)):
    img = read(images[i],1)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)
    for (x,y,w,h) in faces:
        img2 = img[y:(y+h),x:(x+w),:]
    cv2.imwrite('cropped_faces/'+str(i)+'.jpg', \
                cv2.cvtColor(img2, cv2.COLOR_RGB2BGR))
```

A sample of the cropped faces is as follows:

Figure 12.8: Cropped male and female faces

Note that by cropping and keeping the faces only, we are retaining only the information that we want to generate. This way, we are reducing the complexities that the DCGAN would have to learn about.

3. Specify the transformation to perform on each image:

```
transform=transforms.Compose([
                                transforms.Resize(64),
                                transforms.CenterCrop(64),
                                transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
```

4. Define the `Faces` dataset class:

```
class Faces(Dataset):
    def __init__(self, folder):
        super().__init__()
        self.folder = folder
        self.images = sorted(Glob(folder))
    def __len__(self):
        return len(self.images)
    def __getitem__(self, ix):
        image_path = self.images[ix]
        image = Image.open(image_path)
        image = transform(image)
        return image
```

5. Create the dataset object: `ds`:

```
ds = Faces(folder='cropped_faces/')
```

6. Define the `dataloader` class as follows:

```
dataloader = DataLoader(ds, batch_size=64, shuffle=True, num_workers=8)
```

4. Define weight initialization so that the weights have a smaller spread as mentioned in the details of the adversarial training section at **https://arxiv.org/pdf/1511.06434**:

```
def weights_init(m):
    classname = m.__class__.__name__
```

```
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)
```

5. Define the `Discriminator` model class, which takes an image of a shape of batch size x 3 x 64 x 64 and predicts whether it is real or fake:

```
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(
            nn.Conv2d(3,64,4,2,1,bias=False),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Conv2d(64,64*2,4,2,1,bias=False),
            nn.BatchNorm2d(64*2),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Conv2d(64*2,64*4,4,2,1,bias=False),
            nn.BatchNorm2d(64*4),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Conv2d(64*4,64*8,4,2,1,bias=False),
            nn.BatchNorm2d(64*8),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Conv2d(64*8,1,4,1,0,bias=False),
            nn.Sigmoid()
        )
        self.apply(weights_init)
    def forward(self, input): return self.model(input)
```

Obtain a summary of the defined model:

```
!pip install torch_summary
from torchsummary import summary
discriminator = Discriminator().to(device)
summary(discriminator,torch.zeros(1,3,64,64));
```

The preceding code generates the following output:

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
├─Sequential: 1-1                        [-1, 1, 1, 1]             --
|    └─Conv2d: 2-1                       [-1, 64, 32, 32]          3,072
|    └─LeakyReLU: 2-2                    [-1, 64, 32, 32]          --
|    └─Conv2d: 2-3                       [-1, 128, 16, 16]         131,072
|    └─BatchNorm2d: 2-4                  [-1, 128, 16, 16]         256
|    └─LeakyReLU: 2-5                    [-1, 128, 16, 16]         --
|    └─Conv2d: 2-6                       [-1, 256, 8, 8]           524,288
|    └─BatchNorm2d: 2-7                  [-1, 256, 8, 8]           512
|    └─LeakyReLU: 2-8                    [-1, 256, 8, 8]           --
|    └─Conv2d: 2-9                       [-1, 512, 4, 4]           2,097,152
|    └─BatchNorm2d: 2-10                 [-1, 512, 4, 4]           1,024
|    └─LeakyReLU: 2-11                   [-1, 512, 4, 4]           --
|    └─Conv2d: 2-12                      [-1, 1, 1, 1]             8,192
|    └─Sigmoid: 2-13                     [-1, 1, 1, 1]             --
==========================================================================================
Total params: 2,765,568
Trainable params: 2,765,568
Non-trainable params: 0
Total mult-adds (M): 106.58
==========================================================================================
```

Figure 12.9: Summary of the discriminator architecture

6. Define the `Generator` model class that generates fake images from an input of shape batch size x 100 x 1 x 1:

```
class Generator(nn.Module):
    def __init__(self):
        super(Generator,self).__init__()
        self.model = nn.Sequential(
            nn.ConvTranspose2d(100,64*8,4,1,0,bias=False,),
            nn.BatchNorm2d(64*8),
            nn.ReLU(True),
            nn.ConvTranspose2d(64*8,64*4,4,2,1,bias=False),
            nn.BatchNorm2d(64*4),
            nn.ReLU(True),
            nn.ConvTranspose2d( 64*4,64*2,4,2,1,bias=False),
            nn.BatchNorm2d(64*2),
            nn.ReLU(True),
            nn.ConvTranspose2d( 64*2,64,4,2,1,bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            nn.ConvTranspose2d( 64,3,4,2,1,bias=False),
            nn.Tanh()
        )
        self.apply(weights_init)
    def forward(self,input): return self.model(input)
```

Obtain a summary of the defined model:

```
generator = Generator().to(device)
summary(generator,torch.zeros(1,100,1,1))
```

The preceding code generates the following output:

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
├─Sequential: 1-1                        [-1, 3, 64, 64]           --
|    └─ConvTranspose2d: 2-1              [-1, 512, 4, 4]           819,200
|    └─BatchNorm2d: 2-2                  [-1, 512, 4, 4]           1,024
|    └─ReLU: 2-3                         [-1, 512, 4, 4]           --
|    └─ConvTranspose2d: 2-4              [-1, 256, 8, 8]           2,097,152
|    └─BatchNorm2d: 2-5                  [-1, 256, 8, 8]           512
|    └─ReLU: 2-6                         [-1, 256, 8, 8]           --
|    └─ConvTranspose2d: 2-7              [-1, 128, 16, 16]         524,288
|    └─BatchNorm2d: 2-8                  [-1, 128, 16, 16]         256
|    └─ReLU: 2-9                         [-1, 128, 16, 16]         --
|    └─ConvTranspose2d: 2-10             [-1, 64, 32, 32]          131,072
|    └─BatchNorm2d: 2-11                 [-1, 64, 32, 32]          128
|    └─ReLU: 2-12                        [-1, 64, 32, 32]          --
|    └─ConvTranspose2d: 2-13             [-1, 3, 64, 64]           3,072
|    └─Tanh: 2-14                        [-1, 3, 64, 64]           --
==========================================================================================
Total params: 3,576,704
Trainable params: 3,576,704
Non-trainable params: 0
Total mult-adds (M): 431.92
==========================================================================================
```

Figure 12.10: Summary of the generator architecture

Note that we have leveraged `ConvTranspose2d` to gradually upsample an array so that it closely resembles an image.

7. Define the functions to train the generator (`generator_train_step`) and the discriminator (`discriminator_train_step`):

```
def discriminator_train_step(real_data, fake_data):
    d_optimizer.zero_grad()
    prediction_real = discriminator(real_data)
    error_real = loss(prediction_real.squeeze(), \
                      torch.ones(len(real_data)).to(device))
    error_real.backward()
    prediction_fake = discriminator(fake_data)
    error_fake = loss(prediction_fake.squeeze(), \
                      torch.zeros(len(fake_data)).to(device))
    error_fake.backward()
```

```
    d_optimizer.step()
    return error_real + error_fake
def generator_train_step(fake_data):
    g_optimizer.zero_grad()
    prediction = discriminator(fake_data)
    error = loss(prediction.squeeze(), \
                 torch.ones(len(real_data)).to(device))
    error.backward()
    g_optimizer.step()
    return error
```

In the preceding code, we are performing a `.squeeze` operation on top of the prediction as the output of the model has a shape of batch size x 1 x 1 x 1 and it needs to be compared to a tensor that has a shape of batch size x 1.

8. Create the generator and discriminator model objects, the optimizers, and the loss function of the discriminator to be optimized:

```
discriminator = Discriminator().to(device)
generator = Generator().to(device)
loss = nn.BCELoss()
d_optimizer = optim.Adam(discriminator.parameters(), \
                         lr=0.0002, betas=(0.5, 0.999))
g_optimizer = optim.Adam(generator.parameters(), \
                         lr=0.0002, betas=(0.5, 0.999))
```

9. Run the models over increasing epochs, as follows:
    1. Loop through 25 epochs over the `dataloader` function defined in *step 3*:

```
log = Report(25)
for epoch in range(25):
    N = len(dataloader)
    for i, images in enumerate(dataloader):
```

    2. Load real data (`real_data`) and generate fake data (`fake_data`) by passing through the generator network:

```
        real_data = images.to(device)
        fake_data = generator(torch.randn(len(real_data), 100, 1, \
                                          1).to(device)).to(device)
        fake_data = fake_data.detach()
```

Note that the major difference between vanilla GANs and DCGANs when generating `real_data` is that we did not have to flatten `real_data` in the case of DCGANs as we are leveraging CNNs.

3. Train the discriminator using the `discriminator_train_step` function defined in *step 7*:

```
d_loss=discriminator_train_step(real_data, fake_data)
```

4. Generate a new set of images (`fake_data`) from the noisy data (`torch.randn(len(real_data))`) and train the generator using the `generator_train_step` function defined in *step 7*:

```
fake_data = generator(torch.randn(len(real_data), \
                    100, 1, 1).to(device)).to(device)
g_loss = generator_train_step(fake_data)
```

5. Record the losses:

```
    log.record(epoch+(1+i)/N, d_loss=d_loss.item(), \
                   g_loss=g_loss.item(), end='\r')
  log.report_avgs(epoch+1)
log.plot_epochs(['d_loss','g_loss'])
```

The preceding code generates the following output (you can refer to the digital version of the book for the colored image):

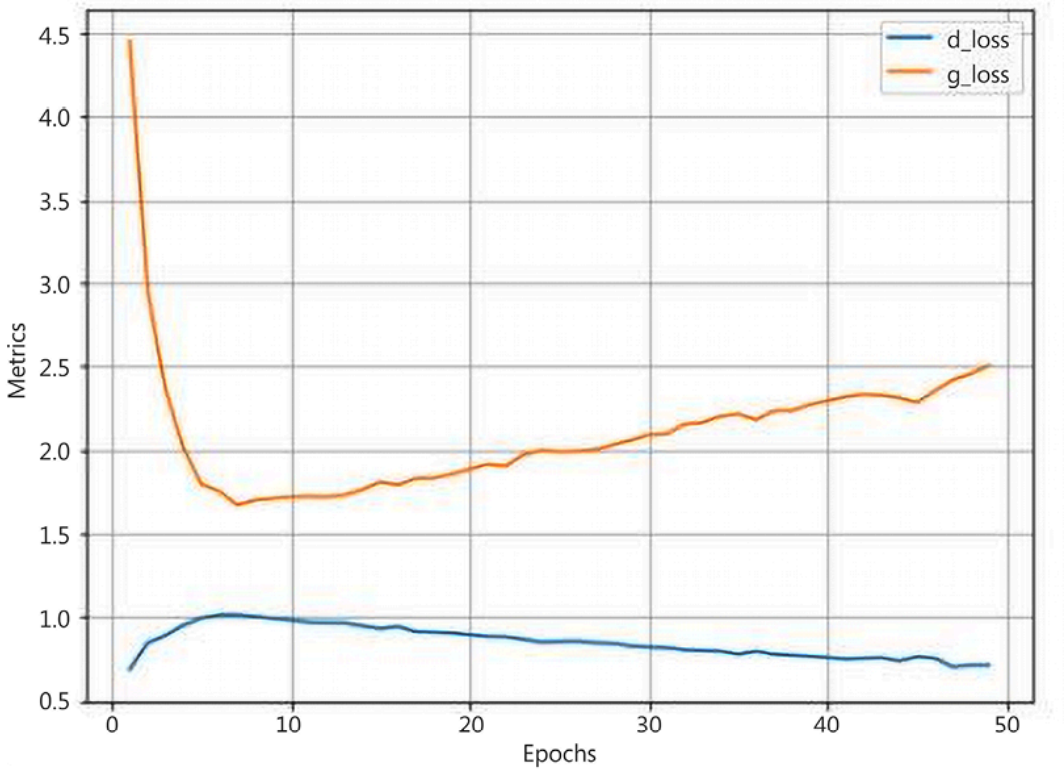


Figure 12.11: Discriminator and generator loss over increasing epochs

Note that in this setting, the variation in generator and discriminator losses does not follow the pattern that we have seen in the case of hand-written digit generation on account of the following:

- We are dealing with bigger images (images that are 64 x 64 x 3 in shape when compared to images of 28 x 28 x 1 shape, which we saw in the previous section).
- Digits have fewer variations when compared to the features that are present in the image of a face.
- Information in handwritten digits is available in only a minority of pixels when compared to the information in images of a face.

Once the training process is complete, generate a sample of images using the following code:

```
generator.eval()
noise = torch.randn(64, 100, 1, 1, device=device)
sample_images = generator(noise).detach().cpu()
```

```
grid = vutils.make_grid(sample_images,nrow=8,normalize=True)
show(grid.cpu().detach().permute(1,2,0), sz=10, \
     title='Generated images')
```

The preceding code generates the following set of images:

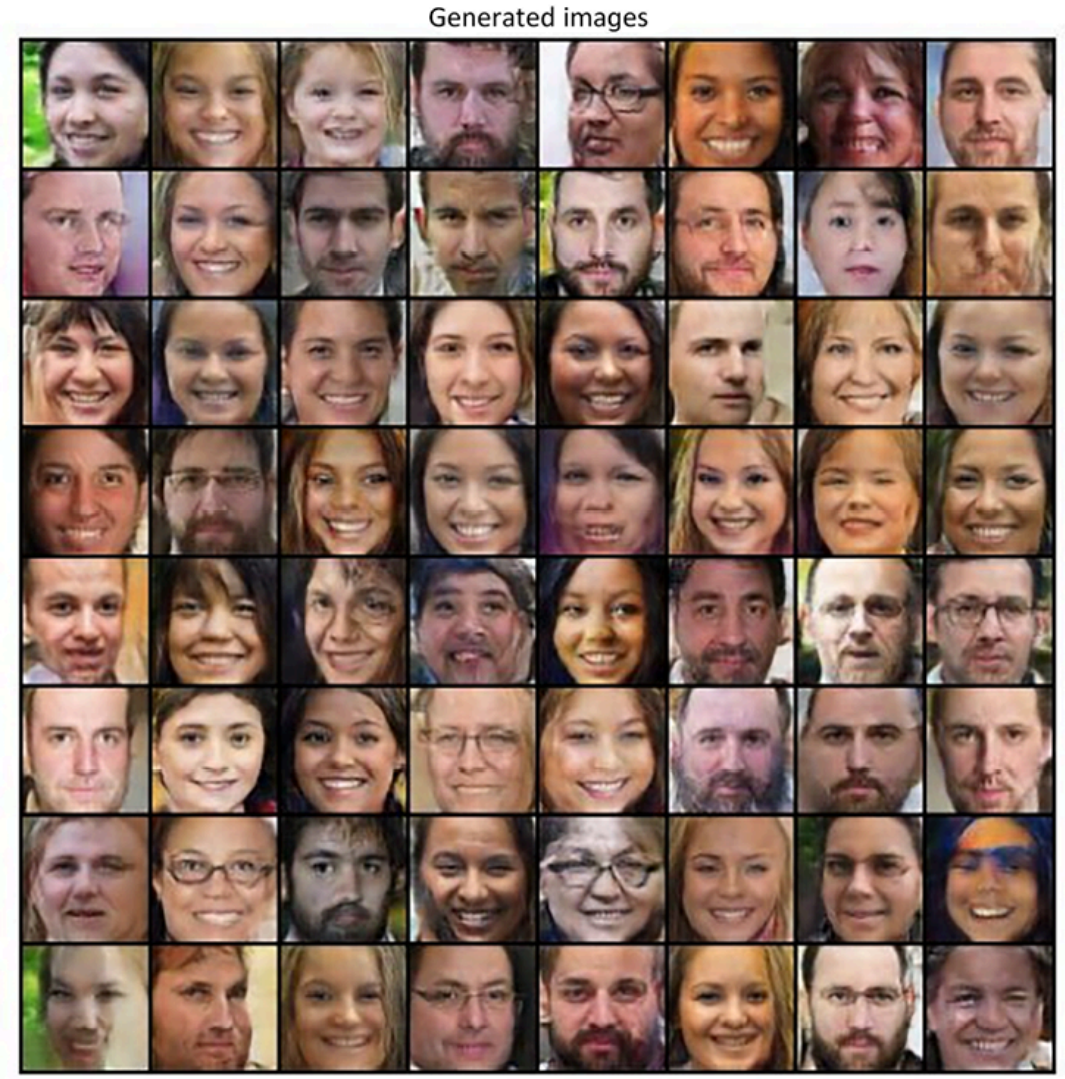


Figure 12.12: Generated images from the trained DCGAN model

Note that while the generator generated images of a face from random noise, the images are decent but still not sufficiently realistic. One potential reason is that not all input images have the same face alignment. As an exercise, we suggest you train the DCGAN only on those images where there is no tilted face and the person is looking straight into the camera in the original image.

In addition, we suggest you try and contrast the generated images with high discriminator scores to the ones with low discriminator scores.

In this section, we have learned about generating images of a face. However, we cannot specify the generation of an image that is of interest to us (for example, a man with a beard). In the next section, we will work toward generating images of a specific class.

Implementing conditional GANs

Imagine a scenario where we want to generate an image of a class of our interest; for example, an image of a cat, a dog, or a man with spectacles. How do we specify that we want to generate an image of interest to us? **Conditional GANs** come to the rescue in this scenario. For now, let's assume that we have the images of male and female faces only, along with their corresponding labels. In this section, we will learn about generating images of a specified class of interest from random noise.

The strategy we adopt is as follows:

1. Specify the label of the image we want to generate as a one-hot-encoded version.
2. Pass the label through an embedding layer to generate a multi-dimensional representation of each class.
3. Generate random noise and concatenate with the embedding layer generated in the previous step.
4. Train the model just like we did in the previous sections, but this time with the noise vector concatenated with the embedding of the class of image we wish to generate.

In the following code, we will code up the preceding strategy:

The following code is available as `Face_generation_using_Conditional_GAN.ipynb` in the `Chapter12` folder in this book's GitHub repository: **https://bit.ly/mcvp-2e**. We strongly recommend you execute the notebook in GitHub to reproduce the results while

you understand the steps to perform and the explanation of various code components from the text.

1. Import the images and the relevant packages:

```
!wget https://www.dropbox.com/s/rbajpdlh7efkdo1/male_female_face_images.zi
!unzip male_female_face_images.zip
!pip install -q --upgrade torch_snippets
from torch_snippets import *
device = "cuda" if torch.cuda.is_available() else "cpu"
from torchvision.utils import make_grid
from torch_snippets import *
from PIL import Image
import torchvision
from torchvision import transforms
import torchvision.utils as vutils
```

2. Create the dataset and dataloader, as follows:
    1. Store the male and female image paths:

```
female_images = Glob('/content/females/*.jpg')
male_images = Glob('/content/males/*.jpg')
```

    2. Ensure that we crop the images so that we retain only faces and discard additional details in an image. First, we will download the cascade filter (more on cascade filters can be found in the *Using OpenCV Utilities for Image Analysis* PDF on GitHub, which will help in identifying faces within an image:

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + \
                        'haarcascade_frontalface_default.xml')
```

    3. Create two new folders (one corresponding to male and another for female images) and dump all the cropped face images into the respective folders:

```
!mkdir cropped_faces_females
!mkdir cropped_faces_males
def crop_images(folder):
    images = Glob(folder+'/*.jpg')
    for i in range(len(images)):
        img = read(images[i],1)
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        faces = face_cascade.detectMultiScale(gray, 1.3, 5)
        for (x,y,w,h) in faces:
            img2 = img[y:(y+h),x:(x+w),:]
            cv2.imwrite('cropped_faces_'+folder+'/'+ \
                    str(i)+'.jpg',cv2.cvtColor(img2, cv2.COLOR_RGB2BGR))
crop_images('females')
crop_images('males')
```

4. Specify the transformation to perform on each image:

```
transform=transforms.Compose([
                                transforms.Resize(64),
                                transforms.CenterCrop(64),
                                transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
```

5. Create the `Faces` dataset class that returns the image and the corresponding gender of the person in it:

```
class Faces(Dataset):
    def __init__(self, folders):
        super().__init__()
        self.folderfemale = folders[0]
        self.foldermale = folders[1]
        self.images = sorted(Glob(self.folderfemale)) + \
                        sorted(Glob(self.foldermale))
    def __len__(self):
        return len(self.images)
    def __getitem__(self, ix):
        image_path = self.images[ix]
        image = Image.open(image_path)
        image = transform(image)
```

```
        gender = np.where('female' in image_path,1,0)
        return image, torch.tensor(gender).long()
```

6. Define the `ds` dataset and `dataloader`:

```
ds = Faces(folders=['cropped_faces_females', \
                    'cropped_faces_males'])
dataloader = DataLoader(ds, batch_size=64, \
                        shuffle=True, num_workers=8)
```

3. Define the weight initialization method (just like we did in the *Using DCGANs to generate face images* section) so that we do not have a widespread variation across randomly initialized weight values:

```
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)
```

4. Define the `Discriminator` model class as follows:
    1. Define the model architecture:

```
class Discriminator(nn.Module):
    def __init__(self, emb_size=32):
        super(Discriminator, self).__init__()
        self.emb_size = 32
        self.label_embeddings = nn.Embedding(2, self.emb_size)
        self.model = nn.Sequential(
            nn.Conv2d(3,64,4,2,1,bias=False),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Conv2d(64,64*2,4,2,1,bias=False),
            nn.BatchNorm2d(64*2),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Conv2d(64*2,64*4,4,2,1,bias=False),
            nn.BatchNorm2d(64*4),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Conv2d(64*4,64*8,4,2,1,bias=False),
```

```
            nn.BatchNorm2d(64*8),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Conv2d(64*8,64,4,2,1,bias=False),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Flatten()
        )
        self.model2 = nn.Sequential(
            nn.Linear(288,100),
            nn.LeakyReLU(0.2,inplace=True),
            nn.Linear(100,1),
            nn.Sigmoid()
        )
        self.apply(weights_init)
```

Note that in the model class, we have an additional parameter, `emb_size`, present in conditional GANs and not in DCGANs. `emb_size` represents the number of embeddings into which we convert the input class label (the class of image we want to generate), which is stored as `label_embeddings`. The reason we convert the input class label from a one-hot-encoded version to embeddings of a higher dimension is that the model has a higher degree of freedom to learn and adjust to deal with different classes.
While the model class, to a large extent, remains the same as what we have seen in DCGANs, we are initializing another model (`model2`) that does the classification exercise. There will be more about how the second model helps after we discuss the `forward` method next. You will also understand the reason why `self.model2` has 288 values as input after you go through the following `forward` method and the summary of the model.

2. Define the `forward` method that takes the image and the label of the image as input:

```
def forward(self, input, labels):
    x = self.model(input)
    y = self.label_embeddings(labels)
    input = torch.cat([x, y], 1)
```

```
        final_output = self.model2(input)
        return final_output
```

In the `forward` method defined, we are fetching the output of the first model (`self.model(input)`) and the output of passing `labels` through `label_embeddings` and then concatenating the outputs. Next, we are passing the concatenated outputs through the second model (`self.model2`) we have defined earlier that fetches us the discriminator output.

3. Obtain the summary of the defined model:

```
!pip install torch_summary
from torchsummary import summary
discriminator = Discriminator().to(device)
summary(discriminator,torch.zeros(32,3,64,64).to(device), \
        torch.zeros(32).long().to(device));
```

The preceding code generates the following output:

```
==========================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================
├─Sequential: 1-1                        [-1, 256]                 --
|    └─Conv2d: 2-1                       [-1, 64, 32, 32]          3,072
|    └─LeakyReLU: 2-2                    [-1, 64, 32, 32]          --
|    └─Conv2d: 2-3                       [-1, 128, 16, 16]         131,072
|    └─BatchNorm2d: 2-4                  [-1, 128, 16, 16]         256
|    └─LeakyReLU: 2-5                    [-1, 128, 16, 16]         --
|    └─Conv2d: 2-6                       [-1, 256, 8, 8]           524,288
|    └─BatchNorm2d: 2-7                  [-1, 256, 8, 8]           512
|    └─LeakyReLU: 2-8                    [-1, 256, 8, 8]           --
|    └─Conv2d: 2-9                       [-1, 512, 4, 4]           2,097,152
|    └─BatchNorm2d: 2-10                 [-1, 512, 4, 4]           1,024
|    └─LeakyReLU: 2-11                   [-1, 512, 4, 4]           --
|    └─Conv2d: 2-12                      [-1, 64, 2, 2]            524,288
|    └─BatchNorm2d: 2-13                 [-1, 64, 2, 2]            128
|    └─LeakyReLU: 2-14                   [-1, 64, 2, 2]            --
|    └─Flatten: 2-15                     [-1, 256]                 --
├─Embedding: 1-2                         [-1, 32]                  64
├─Sequential: 1-3                        [-1, 1]                   --
|    └─Linear: 2-16                      [-1, 100]                 28,900
|    └─LeakyReLU: 2-17                   [-1, 100]                 --
|    └─Linear: 2-18                      [-1, 1]                   101
|    └─Sigmoid: 2-19                     [-1, 1]                   --
==========================================================================
Total params: 3,310,857
Trainable params: 3,310,857
Non-trainable params: 0
Total mult-adds (M): 109.25
```

Figure 12.13: Summary of the discriminator architecture

Note that `self.model2` takes an input of 288 values as the output of `self.model` has 256 values per data point, which is then concatenated with the 32 embedding values of the input class label, resulting in 256 + 32 = 288 input values to `self.model2`.

5. Define the `Generator` network class:
    1. Define the `__init__` method:

```
class Generator(nn.Module):
    def __init__(self, emb_size=32):
        super(Generator,self).__init__()
        self.emb_size = emb_size
        self.label_embeddings = nn.Embedding(2, self.emb_size)
```

Note that in the preceding code, we are using `nn.Embedding` to convert the 2D input (which is of classes) to a 32-dimensional vector (`self.emb_size`).

```
        self.model = nn.Sequential(
            nn.ConvTranspose2d(100+self.emb_size,\
                               64*8,4,1,0,bias=False),
            nn.BatchNorm2d(64*8),
            nn.ReLU(True),
            nn.ConvTranspose2d(64*8,64*4,4,2,1,bias=False),
            nn.BatchNorm2d(64*4),
            nn.ReLU(True),
            nn.ConvTranspose2d(64*4,64*2,4,2,1,bias=False),
            nn.BatchNorm2d(64*2),
            nn.ReLU(True),
            nn.ConvTranspose2d(64*2,64,4,2,1,bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            nn.ConvTranspose2d(64,3,4,2,1,bias=False),
            nn.Tanh()
        )
```

Note that in the preceding code, we have leveraged `nn.ConvTranspose2d` to upscale toward fetching an image as output.

2. Apply weight initialization:

```
        self.apply(weights_init)
```

3. Define the `forward` method, which takes the noise values (`input_noise`) and input label (`labels`) as input and generates the output of the image:

```
def forward(self,input_noise,labels):
    label_embeddings = self.label_embeddings(labels).view(len(labels), \
                               self.emb_size,1, 1)
    input = torch.cat([input_noise, label_embeddings], 1)
    return self.model(input)
```

4. Obtain a summary of the defined `generator` function:

```
generator = Generator().to(device)
summary(generator,torch.zeros(32,100,1,1).to(device), \
        torch.zeros(32).long().to(device));
```

The preceding code generates the following output:

```
==========================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================
├─Embedding: 1-1                        [-1, 32]                  64
├─Sequential: 1-2                       [-1, 3, 64, 64]           --
|    └─ConvTranspose2d: 2-1             [-1, 512, 4, 4]           1,081,344
|    └─BatchNorm2d: 2-2                 [-1, 512, 4, 4]           1,024
|    └─ReLU: 2-3                        [-1, 512, 4, 4]           --
|    └─ConvTranspose2d: 2-4             [-1, 256, 8, 8]           2,097,152
|    └─BatchNorm2d: 2-5                 [-1, 256, 8, 8]           512
|    └─ReLU: 2-6                        [-1, 256, 8, 8]           --
|    └─ConvTranspose2d: 2-7             [-1, 128, 16, 16]         524,288
|    └─BatchNorm2d: 2-8                 [-1, 128, 16, 16]         256
|    └─ReLU: 2-9                        [-1, 128, 16, 16]         --
|    └─ConvTranspose2d: 2-10            [-1, 64, 32, 32]          131,072
|    └─BatchNorm2d: 2-11                [-1, 64, 32, 32]          128
|    └─ReLU: 2-12                       [-1, 64, 32, 32]          --
|    └─ConvTranspose2d: 2-13            [-1, 3, 64, 64]           3,072
|    └─Tanh: 2-14                       [-1, 3, 64, 64]           --
==========================================================================
Total params: 3,838,912
Trainable params: 3,838,912
Non-trainable params: 0
Total mult-adds (M): 436.38
```

Figure 12.14: Summary of Generator architecture

6. Define a function (`noise`) to generate random noise with 100 values and register it to the device:

```
def noise(size):
    n = torch.randn(size, 100, 1, 1, device=device)
    return n.to(device)
```

7. Define the function to train the discriminator:: `discriminator_train_step`:
    1. The discriminator takes four inputs—real images (`real_data`), real labels (`real_labels`), fake images (`fake_data`), and fake labels (`fake_labels`):

```
def discriminator_train_step(real_data, real_labels, \
                             fake_data, fake_labels):
    d_optimizer.zero_grad()
```

    Here, we are resetting the gradient corresponding to the discriminator.

2. Calculate the loss value corresponding to predictions on the real data (`prediction_real`). The loss value output when `real_data` and `real_labels` are passed through the `discriminator` network is compared with the expected value of `(torch.ones(len(real_data),1).to(device))` to obtain `error_real` before performing backpropagation:

```
prediction_real = discriminator(real_data, real_labels)
error_real = loss(prediction_real, \
                  torch.ones(len(real_data),1).to(device))
error_real.backward()
```

3. Calculate the loss value corresponding to predictions on the fake data (`prediction_fake`). The loss value output when `fake_data` and `fake_labels` are passed through the `discriminator` network is compared with the expected value of `(torch.zeros(len(fake_data),1).to(device))` to obtain `error_fake` before performing backpropagation:

```
prediction_fake = discriminator(fake_data, fake_labels)
error_fake = loss(prediction_fake, \
                  torch.zeros(len(fake_data),1).to(device))
error_fake.backward()
```

4. Update weights and return the loss values:

```
d_optimizer.step()
return error_real + error_fake
```

8. Define the training steps for the generator where we pass the fake images (`fake_data`) along with the fake labels (`fake_labels`) as input:

```
def generator_train_step(fake_data, fake_labels):
    g_optimizer.zero_grad()
    prediction = discriminator(fake_data, fake_labels)
    error = loss(prediction, \
                 torch.ones(len(fake_data), 1).to(device))
    error.backward()
```

```
    g_optimizer.step()
    return error
```

Note that the `generator_train_step` function is similar to `discriminator_train_step`, with the exception that this has an expectation of `torch.ones(len(fake_data),1).to(device))` as output in place of zeros given that we are training the generator.

9. Define the `generator` and `discriminator` model objects, the loss optimizers, and the `loss` function:

```
discriminator = Discriminator().to(device)
generator = Generator().to(device)
loss = nn.BCELoss()
d_optimizer = optim.Adam(discriminator.parameters(), \
                         lr=0.0002, betas=(0.5, 0.999))
g_optimizer = optim.Adam(generator.parameters(), \
                         lr=0.0002, betas=(0.5, 0.999))
fixed_noise = torch.randn(64, 100, 1, 1, device=device)
fixed_fake_labels = torch.LongTensor([0]* (len(fixed_noise)//2) \
                    + [1]*(len(fixed_noise)//2)).to(device)
loss = nn.BCELoss()
n_epochs = 25
img_list = []
```

In the preceding code, `while` defining `fixed_fake_labels`, we are specifying that half of the images correspond to one class (class 0) and the rest to another class (class 1). Additionally, we are defining `fixed_noise`, which will be used to generate images from random noise.

10. Train the model over increasing epochs (`n_epochs`):
    1. Specify the length of `dataloader`:

```
log = Report(n_epochs)
for epoch in range(n_epochs):
    N = len(dataloader)
```

    2. Loop through the batch of images along with their labels:

```
    for bx, (images, labels) in enumerate(dataloader):
```

3. Specify `real_data` and `real_labels`:

```
        real_data, real_labels = images.to(device), labels.to(device)
```

4. Initialize `fake_data` and `fake_labels`:

```
        fake_labels = torch.LongTensor(np.random.randint(0, \
                      2,len(real_data))).to(device)
        fake_data=generator(noise(len(real_data)),fake_labels)
        fake_data = fake_data.detach()
```

5. Train the discriminator using the `discriminator_train_step` function defined in *step 7* to calculate discriminator loss (`d_loss`):

```
        d_loss = discriminator_train_step(real_data, \
                real_labels, fake_data, fake_labels)
```

6. Regenerate fake images (`fake_data`) and fake labels (`fake_labels`) and train the generator using the `generator_train_step` function defined in *step 8* to calculate the generator loss (`g_loss`):

```
        fake_labels = torch.LongTensor(np.random.randint(0, \
                      2,len(real_data))).to(device)
        fake_data = generator(noise(len(real_data)), \
                                    fake_labels).to(device)
        g_loss = generator_train_step(fake_data, fake_labels)
```

7. Log the loss metrics as follows:

```
        pos = epoch + (1+bx)/N
        log.record(pos, d_loss=d_loss.detach(), \
```

```
                g_loss=g_loss.detach(), end='\r')
        log.report_avgs(epoch+1)
```

11. Once we train the model, generate the male and female images:

```
        with torch.no_grad():
            fake = generator(fixed_noise, fixed_fake_labels).detach().cpu()
            imgs = vutils.make_grid(fake, padding=2, \
                                    normalize=True).permute(1,2,0)
            img_list.append(imgs)
            show(imgs, sz=10)
```

In the preceding code, we are passing the noise (`fixed_noise`) and labels (`fixed_fake_labels`) to the generator to fetch the `fake` images, which are as follows at the end of 25 epochs of training the models:

Generated male and female faces

Figure 12.15: Generated male and female faces

From the preceding image, we can see that the first 32 images correspond to male images, while the next 32 correspond to female images, which substantiates the fact that the conditional GANs performed as expected.

Summary

In this chapter, we learned about leveraging two different neural networks to generate new images of handwritten digits using GANs. Next, we generated realistic faces using DCGANs. Finally, we learned about conditional GANs, which help us in generating images of a certain class. Having generated images using different techniques, we could still see that the generated images were not sufficiently realistic. Furthermore, while we generated images by specifying the class of images we want to generate in conditional GANs, we are still not in a position to perform image translation, where we ask to replace one object in the image with another one, with everything else left as is. In addition, we are yet to have an image generation mechanism where the number of classes (styles) to generate is more unsupervised.

In the next chapter, we will learn about generating images that are more realistic using some of the latest variants of GANs. In addition, we will learn about generating images of different styles in a more unsupervised manner.

Questions

1. What happens if the learning rate of generator and discriminator models is high?
2. In a scenario where the generator and discriminator are very well trained, what is the probability of a given image being real?
3. Why do we use `ConvTranspose2d` in generating images?
4. Why do we have embeddings with a high embedding size than the number of classes in conditional GANs?
5. How can we generate images of men with beards?

6. Why do we have Tanh activation in the last layer in the generator and not ReLU or sigmoid?
7. Why did we get realistic images even though we did not denormalize the generated data?
8. What happens if we do not crop faces corresponding to images before training the GAN?
9. Why do the weights of the discriminator not get updated when the training generator is updated (as the `generator_train_step` function involves the discriminator network)?
10. Why do we fetch losses on both real and fake images while training the discriminator but only the loss on fake images while training the generator?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

https://packt.link/modcv